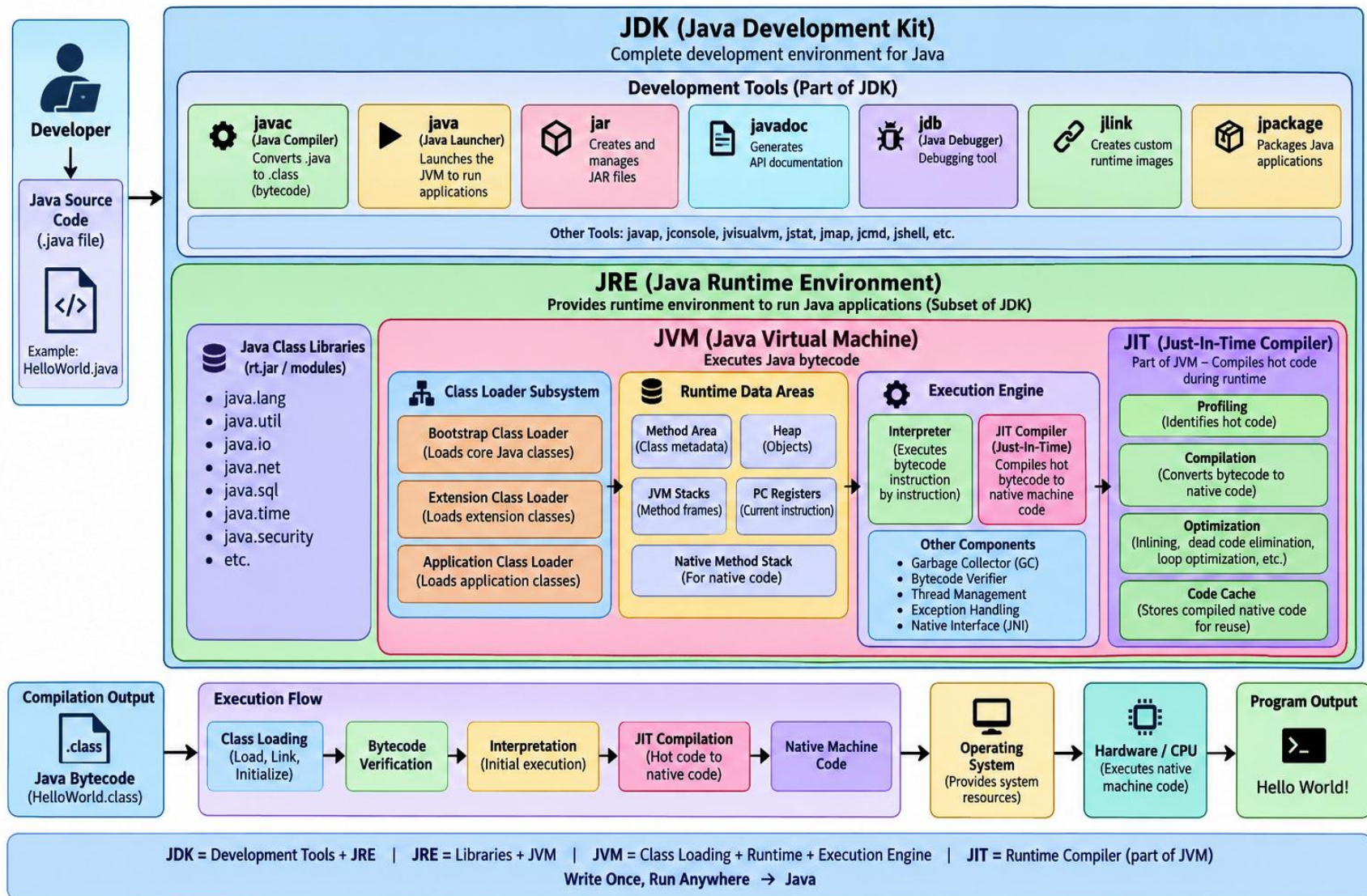
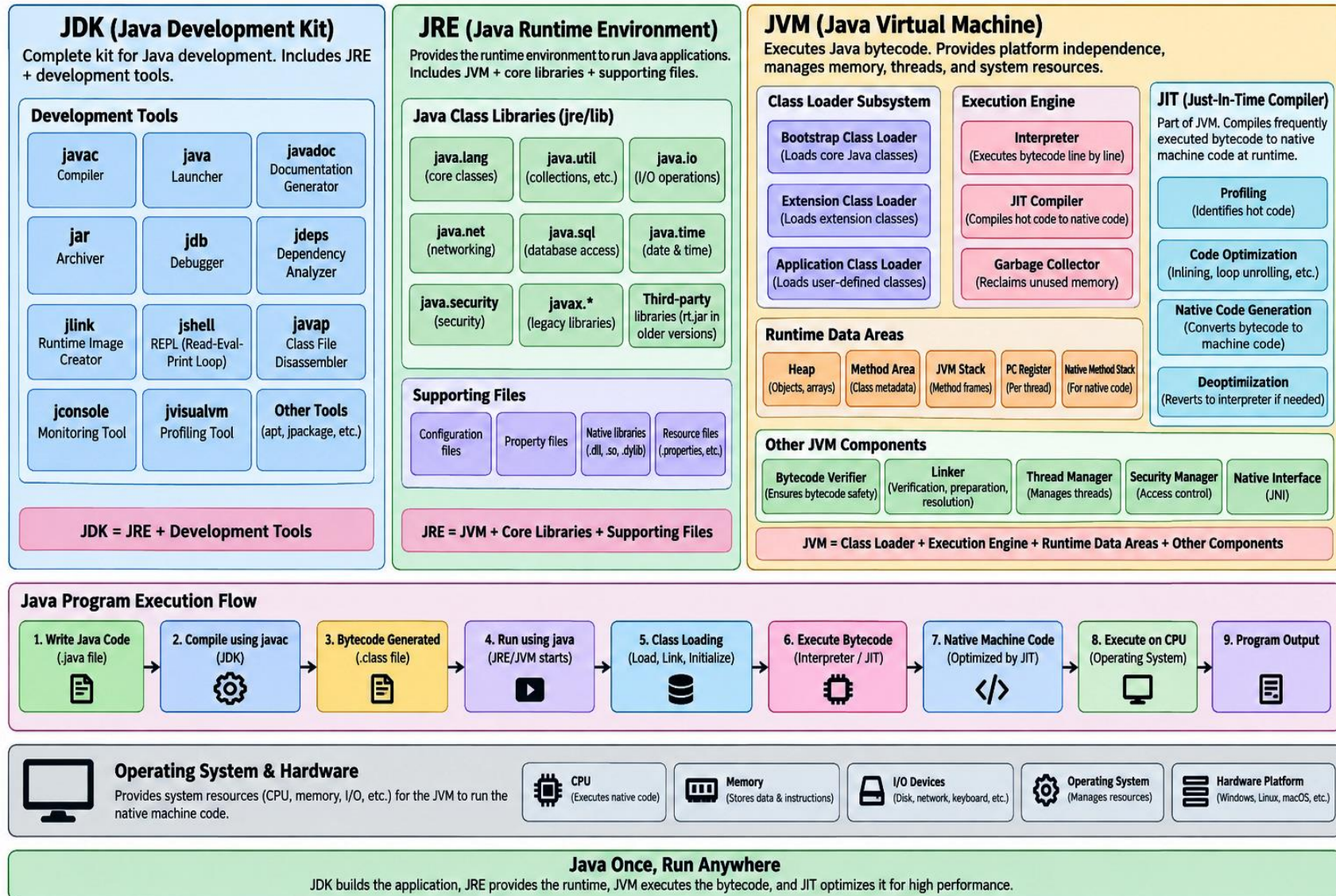


# Java Architecture: JDK, JRE, JVM and JIT (Complete Diagram)

From Source Code to Execution – All Components



# Java Platform Architecture – JDK, JRE, JVM and JIT (Complete View)



## Complete JDK-JRE-JVM-JIT Flow Chart

The easiest way to understand Java execution is to remember:

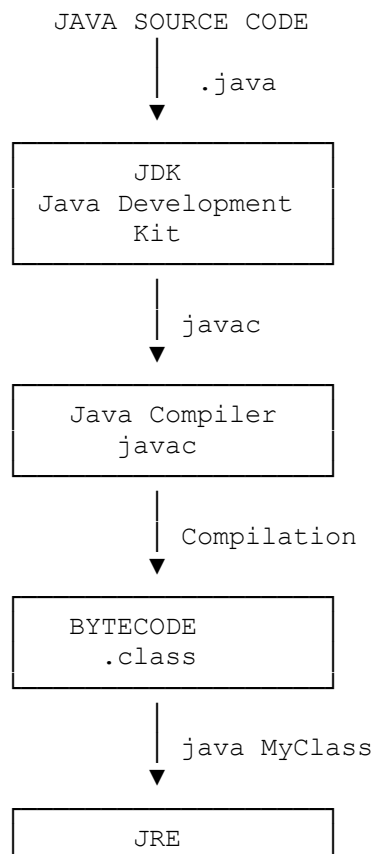
**JDK = Develop + Compile + Run**

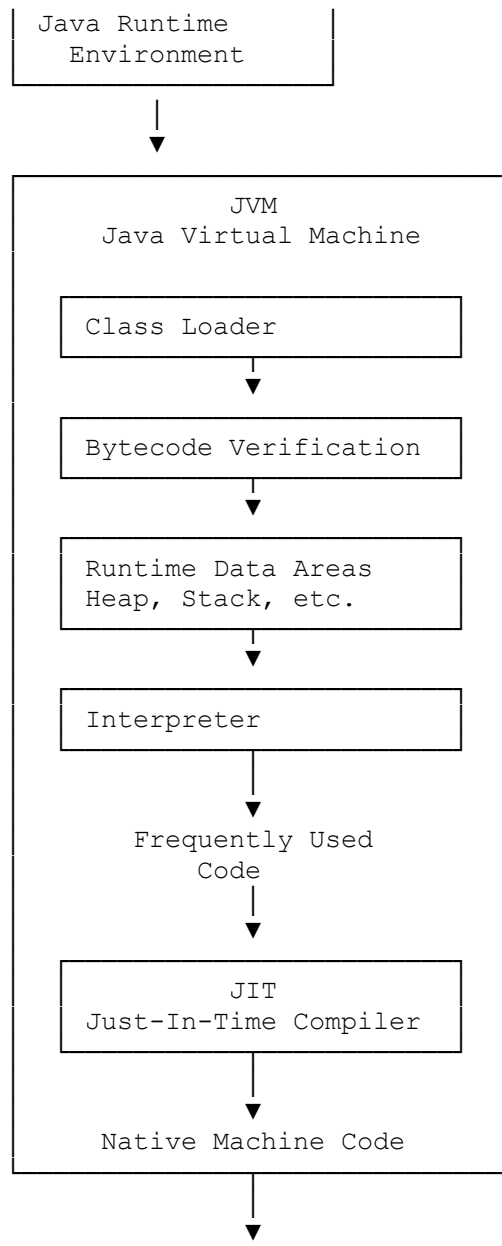
**JRE = Run Java applications**

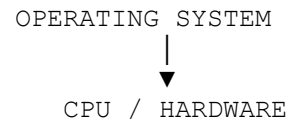
**JVM = Actually executes Java bytecode**

**JIT = Optimizes frequently executed bytecode into native machine code**

### 1. Complete Flow Chart



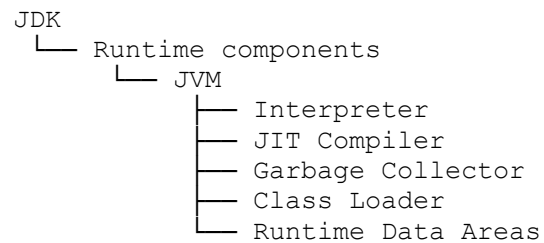




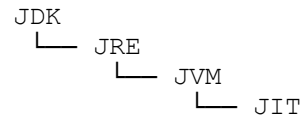
## 2. JDK, JRE, JVM and JIT at a Glance

| Component | Full Form                | Main Purpose                  | Contains                                               | Main User        |
|-----------|--------------------------|-------------------------------|--------------------------------------------------------|------------------|
| JDK       | Java Development Kit     | Develop Java applications     | Compiler + tools + runtime components                  | Java developer   |
| JRE       | Java Runtime Environment | Run Java applications         | JVM + runtime libraries                                | Application user |
| JVM       | Java Virtual Machine     | Execute Java bytecode         | Class loader, execution engine, memory areas, GC, etc. | Runtime          |
| JIT       | Just-In-Time Compiler    | Improve execution performance | Runtime compiler inside JVM                            | JVM              |

### Relationship



Historically, the simplified relationship was:



However, **modern Java distributions do not always package a separately identifiable JRE product**, particularly since Java 9's modular system. So this hierarchy is useful conceptually, but should not be interpreted as the exact packaging of every modern JDK.

---

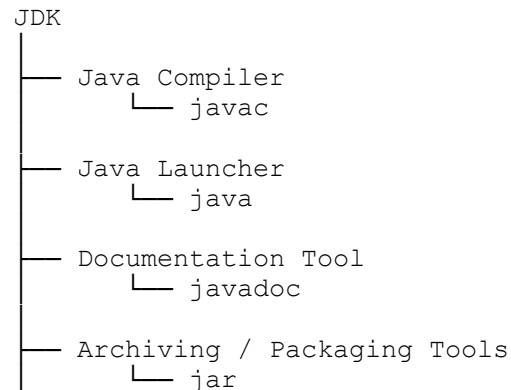
### 3. JDK — Java Development Kit

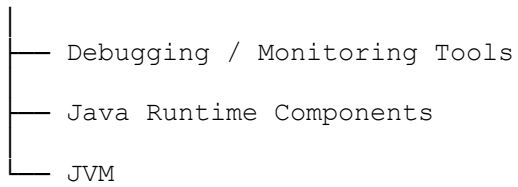
#### Definition

**JDK (Java Development Kit)** is the software development environment used to **develop, compile, test, debug, package, document, and run Java applications**.

A developer normally installs a JDK rather than merely a runtime.

#### Simplified structure





## Example

Suppose you create:

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello Java");  
    }  
}
```

Save it as:

HelloWorld.java

Compile:

```
javac HelloWorld.java
```

This produces:

HelloWorld.class

Run:

```
java HelloWorld
```

The javac compiler is supplied by the **JDK**.

---

#### 4. Important JDK Tools

| Tool            | Purpose                             | Example                 |
|-----------------|-------------------------------------|-------------------------|
| <b>javac</b>    | Compiles Java source code           | javac Hello.java        |
| <b>java</b>     | Launches a Java application         | java Hello              |
| <b>jar</b>      | Creates/manages JAR files           | jar -cf app.jar *.class |
| <b>javadoc</b>  | Generates API documentation         | javadoc Hello.java      |
| <b>javap</b>    | Examines class files                | javap -c Hello          |
| <b>jdb</b>      | Java debugger                       | jdb Hello               |
| <b>jlink</b>    | Creates custom runtime images       | jlink ...               |
| <b>jpackage</b> | Creates native application packages | jpackage ...            |

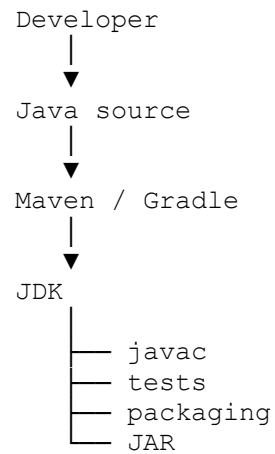
#### Real-time use cases

##### JDK is used when:

- Developing Spring Boot applications
- Developing Android-related Java code
- Building enterprise Java applications
- Compiling Java source code
- Running Maven/Gradle Java builds
- Creating JAR/WAR applications
- Debugging Java applications
- Creating Java libraries

- Running CI/CD builds

For example:



---

## 5. JRE — Java Runtime Environment

### Definition

**JRE (Java Runtime Environment)** is the conceptual environment required to **run Java applications**.

Traditionally:

JRE =  
JVM  
+  
Java Class Libraries  
+  
Supporting Runtime Files

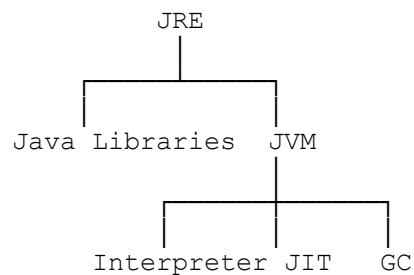
The JRE does **not primarily represent the development toolset**.

For example, a normal Java application needs runtime classes such as:

String  
System  
List  
Map  
File  
Thread

These are supplied through Java's runtime libraries/modules.

### Conceptual diagram



### Important modern-Java clarification

For **Java 8 and earlier**, it was common to download a separate JRE.

From **Java 9 onward**, Java's modular runtime-image model changed how the runtime is packaged. You may encounter a JDK installation and custom runtime images created using jlink, rather than a separately distributed "JRE" installation.

So when learning Java architecture:

**JRE is best understood as the runtime environment/concept, while modern Java distributions may not provide a separate JRE installer.**

---

## 6. JVM — Java Virtual Machine

### Definition

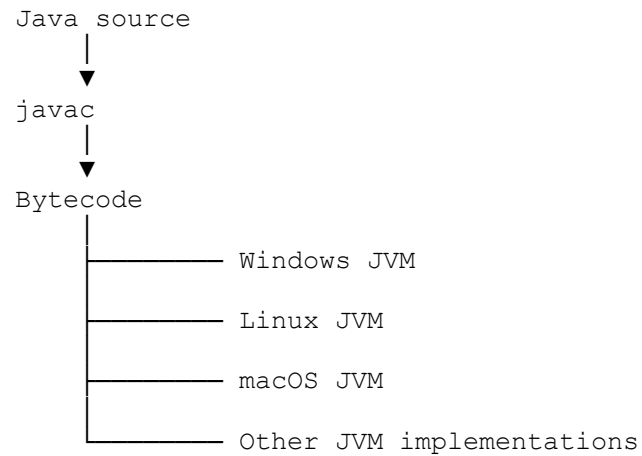
The **JVM (Java Virtual Machine)** is the runtime engine that loads Java bytecode and executes it on a particular operating system and hardware platform.

This is the most important component in Java's:

**"Write Once, Run Anywhere"**

model.

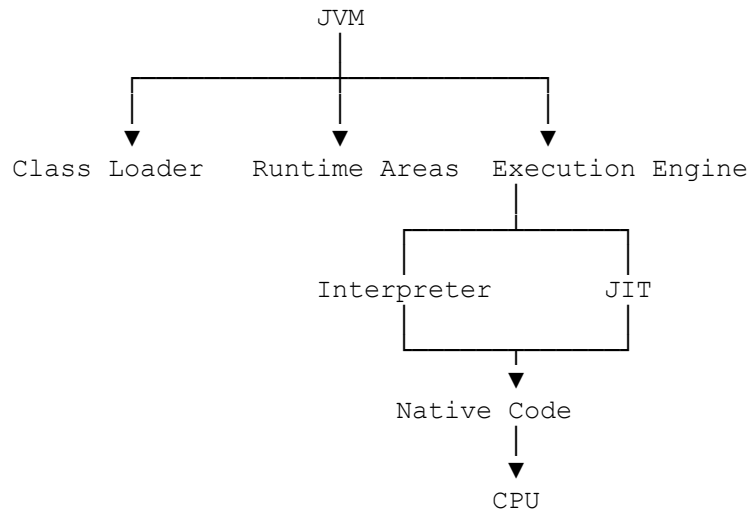
For example:



The same .class bytecode can be executed by a compatible JVM implementation for the target platform.

---

## 7. Major Components Inside the JVM



The JVM also includes important runtime mechanisms such as garbage collection and native-interface support.

---

## 8. Class Loader

### Definition

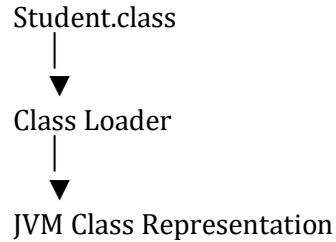
The **Class Loader subsystem** loads class definitions into the JVM when they are needed.

For example:

```
Student student = new Student();
```

When Student is required, the JVM needs to make its class definition available.

Conceptually:



### **Real-time use cases**

Class loading is fundamental to:

- Application startup
- Frameworks
- Plugin systems
- Application servers
- JDBC drivers
- Spring applications
- Modular applications
- Dynamic loading

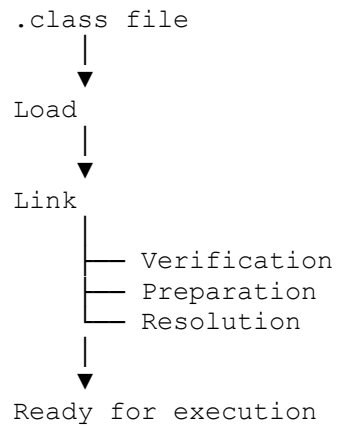
For example, enterprise servers can load application classes separately from server/framework classes.

---

## **9. Bytecode Verification**

After loading, JVM security and correctness checks are performed as part of class loading/linking.

Conceptually:



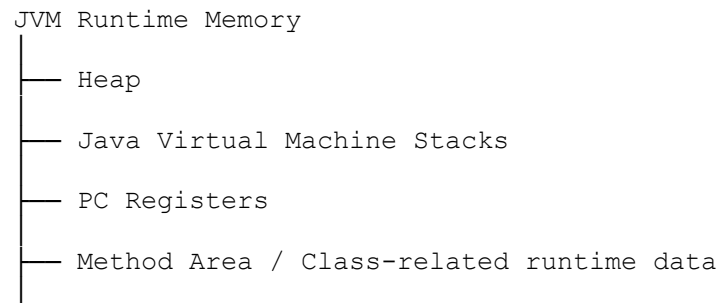
Verification helps ensure that class files conform to JVM constraints and don't violate important runtime safety rules.

---

## 10. Runtime Data Areas

The JVM needs memory areas to execute Java programs.

A simplified view is:



└─ Native Method Stacks

## Heap

Objects are generally allocated in the heap.

Example:

```
Student s = new Student();
```

Conceptually:

Heap

```
Student object  
name = "Anurag"  
age = 43
```

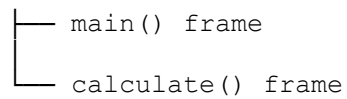
## JVM Stack

Each Java thread has its own JVM stack containing frames for method execution.

```
public static void main(String[] args) {  
    int x = 10;  
    calculate(x);  
}
```

Conceptually:

Thread  
|  
▼  
JVM Stack  
|



## PC Register

Each JVM thread has a program-counter concept used to track the current execution position.

## Method Area / Class-related Runtime Data

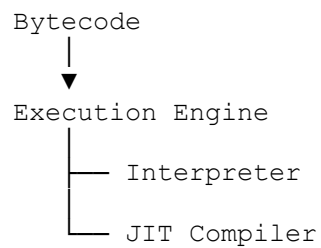
The JVM maintains class metadata and related runtime information. The exact implementation details vary between JVM implementations and Java versions.

---

## 11. Execution Engine

The **Execution Engine** is responsible for executing the loaded bytecode.

Conceptually:



This brings us to the JIT.

---

## 12. JIT — Just-In-Time Compiler

### Definition

**JIT (Just-In-Time Compiler)** is a runtime compiler used by many JVM implementations to compile frequently executed bytecode into optimized **native machine code** while the application is running.

The key idea is:

**Compile important code at runtime instead of interpreting every instruction forever.**

---

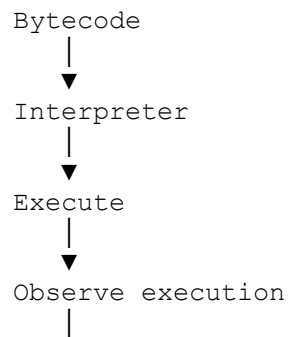
## 13. Interpreter vs JIT

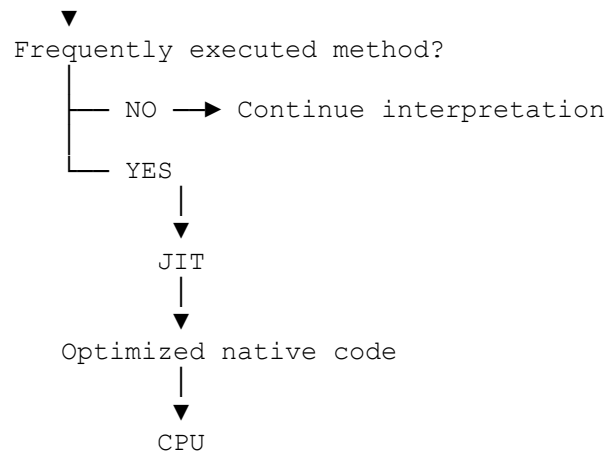
Suppose you have:

```
for (int i = 0; i < 1_000_000; i++) {  
    calculate(i);  
}
```

The JVM initially has bytecode.

Conceptually:





#### 14. Why Does the JVM Need Both Interpreter and JIT?

Because they have different strengths.

| Interpreter                                                 | JIT                                   |
|-------------------------------------------------------------|---------------------------------------|
| <b>Starts executing quickly</b>                             | Usually takes time to compile         |
| <b>Minimal compilation overhead</b>                         | Has compilation overhead              |
| <b>Executes bytecode</b>                                    | Produces native machine code          |
| <b>Useful during startup</b>                                | Very useful for long-running code     |
| <b>Doesn't require extensive profiling before execution</b> | Can use runtime profiling information |
| <b>Generally slower for hot code</b>                        | Generally faster for hot code         |

A JVM can therefore use a **tiered execution strategy** rather than simply choosing one or the other.

---

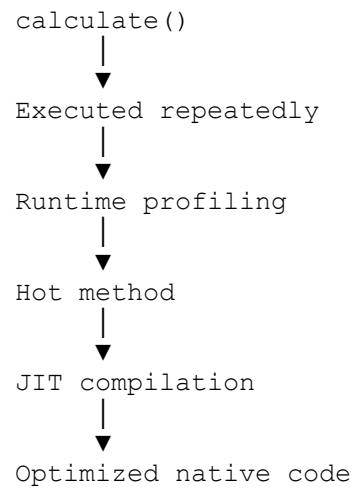
## 15. What Is "Hot Code"?

A method or code path that executes frequently is often called **hot code**.

Example:

```
for (int i = 0; i < 100_000_000; i++) {  
    calculate(i);  
}
```

If `calculate()` is executed millions of times, the JVM can identify it as important and compile/optimize it.



## 16. JIT Optimization Example

Suppose:

```
int square(int x) {  
    return x * x;  
}
```

And the application repeatedly executes:

```
square(10);  
square(20);  
square(30);  
...
```

The JIT may optimize the generated machine code based on runtime information.

Depending on the JVM, compiler, profiling information, and circumstances, optimizations can include:

- Method inlining
- Dead-code elimination
- Loop optimizations
- Constant propagation
- Escape analysis
- Devirtualization
- Other machine-code optimizations

### Method inlining

Instead of repeatedly performing a method call:

```
main()  
└─ square()
```

the compiler may effectively integrate the method's operations into the caller:

```
main()
└── x * x
```

This can reduce method-call overhead and enable further optimizations.

---

## 17. Complete Example

Consider:

```
public class Calculator {

    static int square(int x) {
        return x * x;
    }

    public static void main(String[] args) {

        long total = 0;

        for (int i = 0; i < 10_000_000; i++) {
            total += square(i);
        }

        System.out.println(total);
    }
}
```

### Step 1 — Source Code

Calculator.java

Developer writes Java source.

### Step 2 — JDK

javac Calculator.java

The JDK's Java compiler converts source code into bytecode.

Calculator.java



javac



Calculator.class

### Step 3 — Bytecode

The .class file contains JVM bytecode.

It is **not normally CPU-specific machine code**.

.class

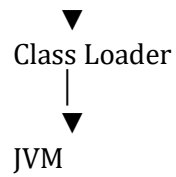


JVM bytecode

### Step 4 — JVM loads the class

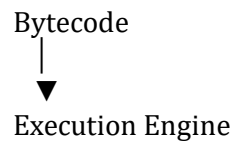
Calculator.class





### **Step 5 — Initial execution**

The JVM can begin execution through its execution mechanisms, including interpretation and/or compilation.



### **Step 6 — Hot code detection**

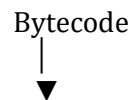
The loop executes millions of times.

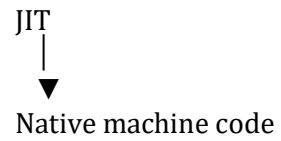
```
square()
square()
square()
square()
...
millions of executions
```

The JVM's runtime profiling identifies frequently executed code.

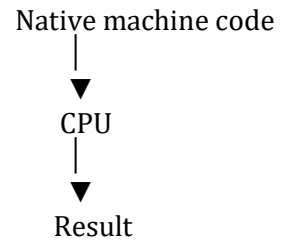
### **Step 7 — JIT compilation**

The JIT compiles suitable hot code into native machine code.

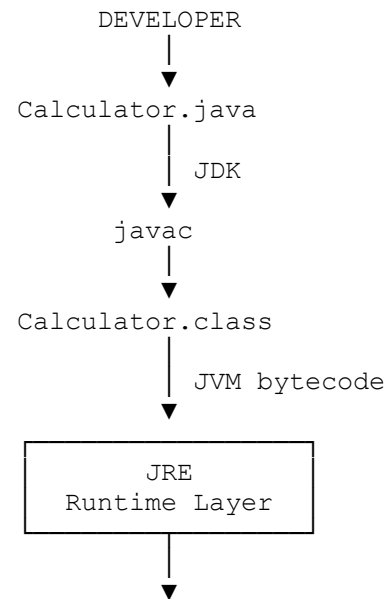


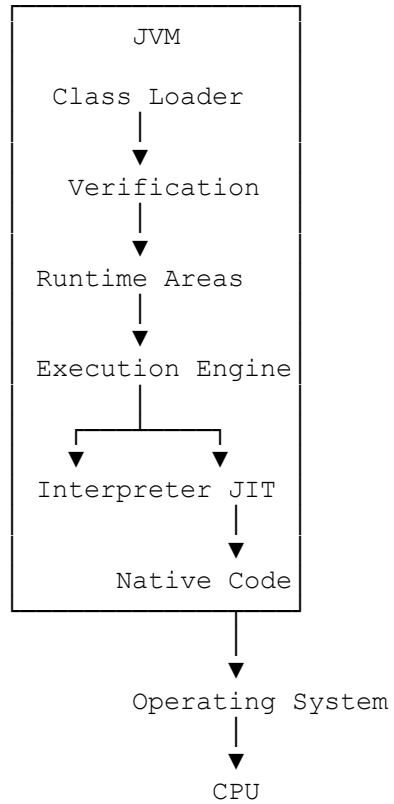


### Step 8 — CPU execution



## 18. Complete End-to-End Architecture





---

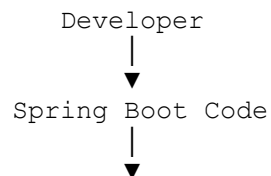
## 19. Real-Time Use Cases

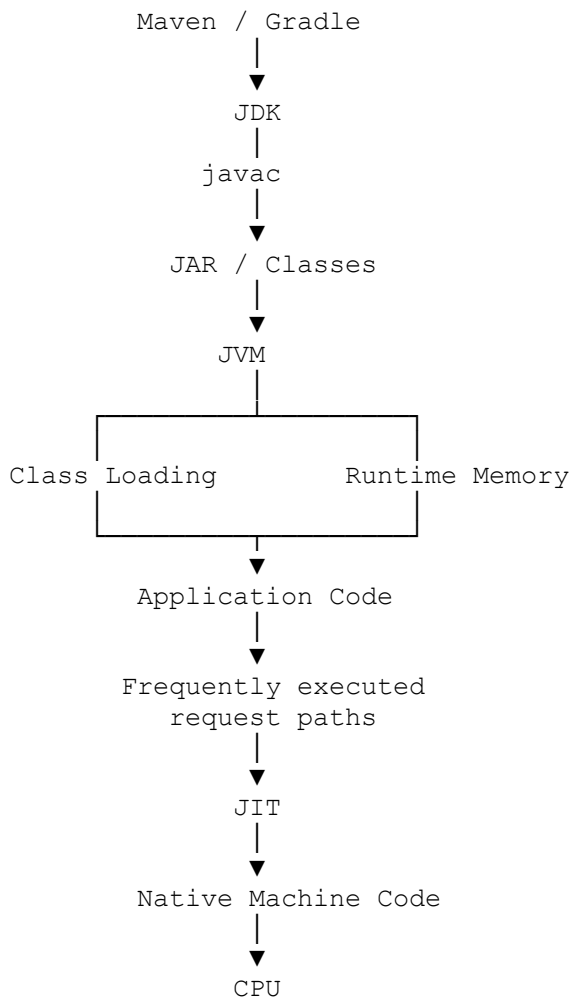
| Component | Real-time use case   | Example                                     |
|-----------|----------------------|---------------------------------------------|
| JDK       | Software development | Developer builds a Spring Boot application  |
| JDK       | Compilation          | javac converts .java → .class               |
| JDK       | Testing              | Maven/Gradle uses a JDK to compile and test |

|                   |                             |                                                 |
|-------------------|-----------------------------|-------------------------------------------------|
| JDK               | Packaging                   | Creating JARs                                   |
| JRE concept       | Application runtime         | Running a Java server/application               |
| JVM               | Bytecode execution          | Running .class files                            |
| JVM Class Loader  | Dynamic class loading       | Loading framework/application classes           |
| JVM Heap          | Object storage              | new Customer()                                  |
| JVM Stack         | Method execution            | Method call frames for each thread              |
| Garbage Collector | Automatic memory management | Reclaiming unreachable objects                  |
| Interpreter       | Early/less-hot execution    | Quickly getting code running                    |
| JIT               | Performance optimization    | Compiling frequently executed methods           |
| JIT               | Long-running servers        | Optimizing heavily used request-processing code |
| JIT               | Computational workloads     | Optimizing repeated calculations                |
| JIT               | Web applications            | Optimizing frequently executed service methods  |

## 20. Example: Spring Boot Application

Consider a real enterprise application:





For a production web application receiving thousands or millions of requests, JIT optimization can become particularly important because the same methods and code paths may execute repeatedly.

---

## 21. JDK vs JRE vs JVM vs JIT

| Question                                 | JDK                 | JRE                         | JVM                                  | JIT                         |
|------------------------------------------|---------------------|-----------------------------|--------------------------------------|-----------------------------|
| <b>What is it?</b>                       | Development kit     | Runtime environment concept | Virtual machine                      | Runtime compiler            |
| <b>Primary job</b>                       | Develop Java        | Run Java                    | Execute bytecode                     | Optimize execution          |
| <b>Compiles .java?</b>                   | <b>Yes</b>          | No                          | No, in the traditional sense         | No source-code compilation  |
| <b>Executes bytecode?</b>                | Through its runtime | Yes                         | <b>Yes</b>                           | Compiles selected bytecode  |
| <b>Contains javac?</b>                   | <b>Yes</b>          | No                          | No                                   | No                          |
| <b>Contains JVM?</b>                     | Runtime included    | <b>Conceptually yes</b>     | Itself                               | No                          |
| <b>Converts bytecode to native code?</b> | Through JVM/JIT     | Through JVM/JIT             | <b>Yes, via execution mechanisms</b> | <b>Yes</b>                  |
| <b>Developer-oriented?</b>               | <b>Yes</b>          | No                          | No                                   | No                          |
| <b>Runtime-oriented?</b>                 | Yes                 | <b>Yes</b>                  | <b>Yes</b>                           | <b>Yes</b>                  |
| <b>Main purpose</b>                      | Build applications  | Provide runtime             | Execute applications                 | Improve runtime performance |

---

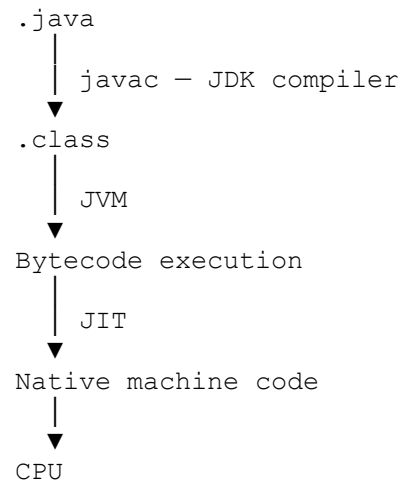
## 22. Most Important Distinction

A common misconception is:

**"JIT compiles Java source code."**

That's incorrect.

The normal pipeline is:



Therefore:

**javac**

Java Source Code → JVM Bytecode

**JIT**

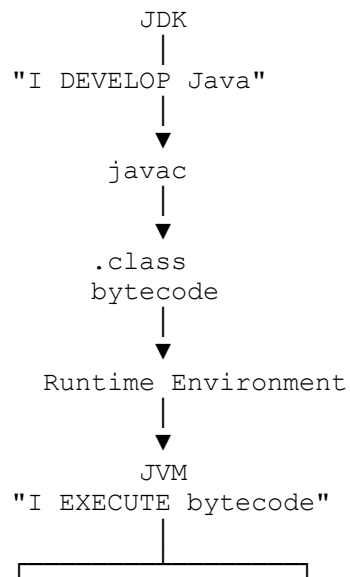
Frequently executed JVM Bytecode → Native Machine Code

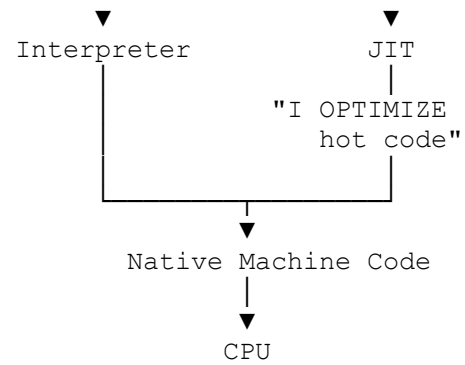
---

## 23. One-Line Definitions

| Component | One-line definition                                                                                                                      |
|-----------|------------------------------------------------------------------------------------------------------------------------------------------|
| JDK       | A complete Java development kit used to build, compile, test, package, debug, and run Java applications.                                 |
| JRE       | The conceptual Java runtime environment containing what is needed to execute Java applications, including the JVM and runtime libraries. |
| JVM       | The virtual machine that loads and executes Java bytecode.                                                                               |
| JIT       | A runtime compiler that converts frequently executed bytecode into optimized native machine code.                                        |

### Final mental model





**In short:**

**JDK builds Java programs → bytecode is produced → the JVM executes that bytecode → the JIT compiles hot bytecode into optimized native code → the CPU executes it.**

### Difference Between JDK, JRE, JVM and JIT

| Aspect                                  | JDK                                                                                                          | JRE                                                                                              | JVM                                                                                  | JIT                                                                                                               |
|-----------------------------------------|--------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| Full Form                               | Java Development Kit                                                                                         | Java Runtime Environment                                                                         | Java Virtual Machine                                                                 | Just-In-Time Compiler                                                                                             |
| Definition                              | A complete development kit used to <b>develop, compile, debug, test, package, and run</b> Java applications. | A runtime environment concept containing the components needed to <b>run Java applications</b> . | A virtual machine that <b>loads, verifies, manages, and executes Java bytecode</b> . | A runtime compiler that <b>compiles frequently executed ("hot") bytecode into optimized native machine code</b> . |
| Primary Purpose                         | <b>Develop Java software</b>                                                                                 | <b>Run Java software</b>                                                                         | <b>Execute Java bytecode</b>                                                         | <b>Improve execution performance</b>                                                                              |
| Main User                               | Java developer                                                                                               | Application/runtime user                                                                         | JVM/runtime itself                                                                   | JVM                                                                                                               |
| Works with                              | .java, .class, JARs, modules, etc.                                                                           | Java runtime libraries and JVM                                                                   | .class bytecode                                                                      | JVM bytecode                                                                                                      |
| Compiles .java source?                  | <b>Yes</b>                                                                                                   | No                                                                                               | No, not source-code compilation                                                      | No                                                                                                                |
| Tool responsible for source compilation | javac                                                                                                        | —                                                                                                | —                                                                                    | —                                                                                                                 |
| Produces .class files?                  | <b>Yes, through javac</b>                                                                                    | No                                                                                               | No                                                                                   | No                                                                                                                |
| Executes bytecode?                      | Through its runtime components                                                                               | <b>Yes</b>                                                                                       | <b>Yes</b>                                                                           | Compiles selected bytecode for execution                                                                          |

|                                       |                                                                 |                                                    |                                                             |                                             |
|---------------------------------------|-----------------------------------------------------------------|----------------------------------------------------|-------------------------------------------------------------|---------------------------------------------|
| Converts source code to bytecode?     | <b>Yes</b>                                                      | No                                                 | No                                                          | No                                          |
| Converts bytecode to native code?     | Through the JVM/JIT                                             | Through the JVM/JIT                                | <b>Yes, through its execution mechanisms</b>                | <b>Yes</b>                                  |
| Contains JVM?                         | A JDK provides the Java runtime/JVM needed to run applications. | Conceptually, yes                                  | It <b>is</b> the virtual machine                            | No; JIT is a component of the JVM           |
| Contains JIT?                         | Through the JVM implementation supplied with the JDK            | Through the JVM implementation                     | <b>Yes, in JVM implementations that use JIT compilation</b> | <b>Itself</b>                               |
| Contains Java libraries?              | <b>Yes</b>                                                      | <b>Yes</b>                                         | Uses/loads runtime libraries and platform classes           | Uses classes/runtime facilities through JVM |
| Contains javac?                       | <b>Yes</b>                                                      | No                                                 | No                                                          | No                                          |
| Contains java launcher?               | <b>Yes</b>                                                      | Historically associated with runtime distributions | The launcher starts the JVM                                 | No                                          |
| Contains jar tool?                    | <b>Yes</b>                                                      | No                                                 | No                                                          | No                                          |
| Contains javadoc?                     | <b>Yes</b>                                                      | No                                                 | No                                                          | No                                          |
| Contains debugging/development tools? | <b>Yes</b>                                                      | No                                                 | Runtime mechanisms only                                     | No                                          |

|                                            |                                                     |                                                            |                                                       |                                                   |
|--------------------------------------------|-----------------------------------------------------|------------------------------------------------------------|-------------------------------------------------------|---------------------------------------------------|
| Memory management                          | Provides the runtime/JVM used for memory management | Runtime environment includes JVM services                  | <b>Manages runtime memory and participates in GC</b>  | Can optimize generated code and runtime execution |
| Garbage Collection                         | Via its JVM/runtime                                 | Via its JVM                                                | <b>Yes</b>                                            | No; JIT is not the garbage collector              |
| Platform dependent?                        | JDK distributions are platform-specific             | Runtime distributions are platform-specific                | JVM implementations are platform-specific             | JIT compiler is JVM/platform-specific             |
| Java bytecode itself platform independent? | —                                                   | —                                                          | <b>Yes, the JVM abstracts the underlying platform</b> | Generated native code is platform-specific        |
| Typical installation                       | JDK 21, JDK 25, etc.                                | Historically a separate JRE, especially Java 8 and earlier | Included as part of a Java runtime/JDK                | Internal component of the JVM                     |
| Example command                            | javac Hello.java                                    | java Hello                                                 | JVM is launched by java                               | Operates internally during JVM execution          |
| Example                                    | Compile a Spring Boot application                   | Run a Java application                                     | Execute Hello.class                                   | Optimize a frequently executed method             |
| Real-time use                              | Software development                                | Application execution                                      | Bytecode execution                                    | High-performance execution                        |
| Developer needs it?                        | <b>Yes</b>                                          | Usually the JDK is installed instead                       | Required to run Java applications                     | Automatically used by the JVM when applicable     |
| Can it run Java applications?              | <b>Yes</b>                                          | <b>Yes</b>                                                 | <b>Yes</b>                                            | Not independently                                 |

|                                   |                                         |                                                                                               |                               |                                        |
|-----------------------------------|-----------------------------------------|-----------------------------------------------------------------------------------------------|-------------------------------|----------------------------------------|
| Can it develop Java applications? | <b>Yes</b>                              | No                                                                                            | No                            | No                                     |
| Can it exist independently?       | Yes, as a Java development distribution | Historically yes; modern Java uses runtime images rather than always providing a separate JRE | JVM is a runtime component    | No; it is part of a JVM implementation |
| Position in execution pipeline    | Development environment                 | Runtime environment                                                                           | Execution engine/environment  | Optimization component inside JVM      |
| Simple description                | <b>Java toolbox</b>                     | <b>Java runtime environment</b>                                                               | <b>Java execution machine</b> | <b>Java runtime optimizer/compiler</b> |

## 1. JDK — Java Development Kit

| Item           | Explanation                                                                                      |
|----------------|--------------------------------------------------------------------------------------------------|
| Definition     | The JDK is the collection of tools and runtime components required to develop Java applications. |
| Main job       | Write → compile → test → debug → package → run Java applications.                                |
| Important tool | Javac                                                                                            |
| Input          | Java source code (.java)                                                                         |
| Output         | JVM bytecode (.class)                                                                            |
| Example        | javac HelloWorld.java                                                                            |

|                    |                                                                                                             |
|--------------------|-------------------------------------------------------------------------------------------------------------|
| Real-time use case | A developer building a banking application with Spring Boot uses a JDK to compile and test the application. |
|--------------------|-------------------------------------------------------------------------------------------------------------|

### Example

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello Java");
    }
}
```

Compile:

```
javac HelloWorld.java
```

The JDK compiler produces:

HelloWorld.class

Conceptually:

HelloWorld.java



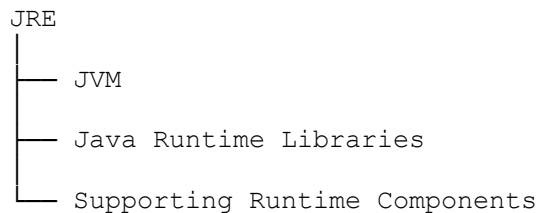
HelloWorld.class

## 2. JRE — Java Runtime Environment

| Item       | Explanation                                                              |
|------------|--------------------------------------------------------------------------|
| Definition | The JRE is the conceptual environment required to run Java applications. |

|                         |                                                                                |
|-------------------------|--------------------------------------------------------------------------------|
| Main job                | Provide the runtime facilities needed by Java applications.                    |
| Contains conceptually   | JVM + Java runtime libraries + supporting runtime components                   |
| Compiles source code?   | No                                                                             |
| Runs Java applications? | Yes                                                                            |
| Example                 | Running java HelloWorld                                                        |
| Real-time use case      | A server or user's machine needs a Java runtime to execute a Java application. |

Conceptually:



### Important modern Java clarification

The term **JRE** is still useful for understanding Java architecture, but modern Java distributions are different from the old Java 8 packaging model.

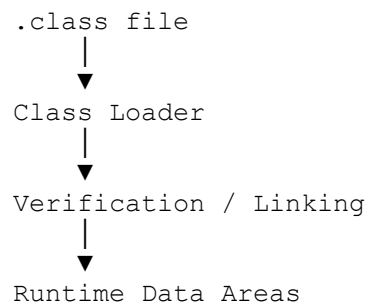
From Java 9 onward, the Java Platform Module System and runtime images changed how Java runtimes are packaged. Modern JDK distributions generally don't require a separately installed traditional "JRE"; custom runtime images can also be created with tools such as [jlink](#).

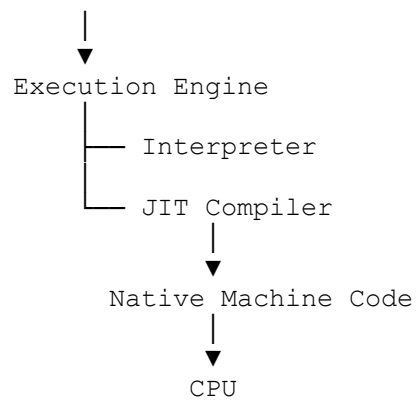
---

### 3. JVM — Java Virtual Machine

| Item               | Explanation                                                                                            |
|--------------------|--------------------------------------------------------------------------------------------------------|
| Definition         | The JVM is the virtual machine responsible for loading and executing Java bytecode.                    |
| Main job           | Execute .class bytecode.                                                                               |
| Input              | JVM bytecode                                                                                           |
| Output             | Execution on the underlying machine                                                                    |
| Contains           | Class loading, runtime memory areas, execution mechanisms, garbage collection, native interfaces, etc. |
| Interpreter        | Can execute bytecode directly                                                                          |
| JIT                | Can compile frequently executed code into native machine code                                          |
| Real-time use case | Running a Java web server, desktop application, enterprise application, or backend service.            |

#### JVM flow





---

#### 4. JIT — Just-In-Time Compiler

| Item                 | Explanation                                                                                                       |
|----------------------|-------------------------------------------------------------------------------------------------------------------|
| Definition           | JIT is a runtime compiler that compiles suitable frequently executed bytecode into optimized native machine code. |
| Main job             | Improve performance during program execution.                                                                     |
| Located in           | JVM implementation                                                                                                |
| Input                | JVM bytecode plus runtime profiling information                                                                   |
| Output               | Native machine code                                                                                               |
| When used?           | Particularly for frequently executed/hot code                                                                     |
| Works independently? | No                                                                                                                |
| Real-time use case   | A long-running Java server repeatedly executing the same request-processing methods.                              |

## Example

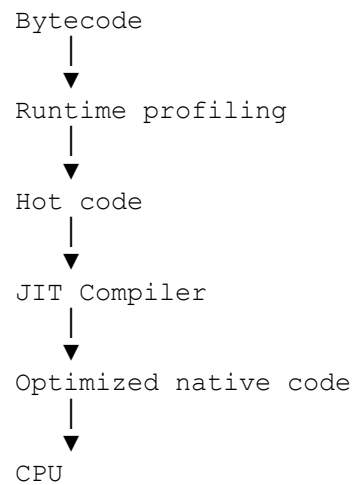
Suppose:

```
static int square(int x) {  
    return x * x;  
}
```

And the application calls it millions of times:

```
square()  
square()  
square()  
square()  
...  
millions of times
```

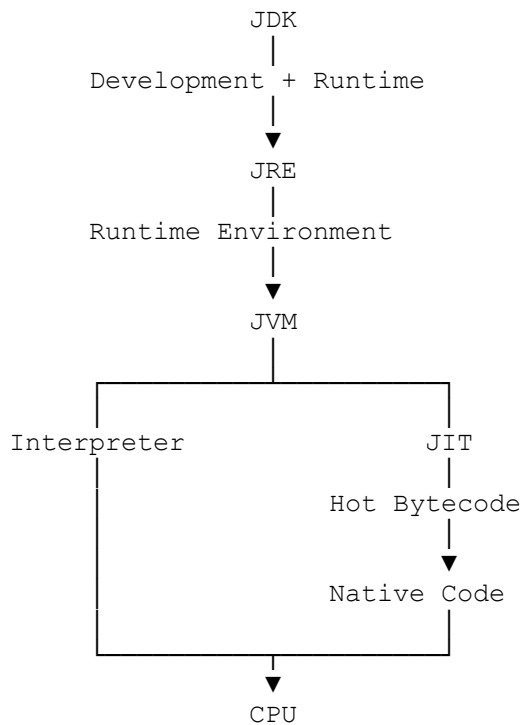
The JVM can identify frequently executed code and compile suitable portions:



## 5. The Most Important Relationship

| Level | Component | Think of it as                              |
|-------|-----------|---------------------------------------------|
| 1     | JDK       | ☐ Complete Java development toolbox         |
| 2     | JRE       | ☐ Java runtime environment                  |
| 3     | JVM       | ☐ Virtual machine executing Java bytecode   |
| 4     | JIT       | ⚡ Runtime compiler/optimizer inside the JVM |

Conceptually:



---

## 6. Complete Java Execution Example

Suppose you write:

```
public class Calculator {  
  
    static int add(int a, int b) {  
        return a + b;  
    }  
  
    public static void main(String[] args) {  
  
        for (int i = 0; i < 10_000_000; i++) {  
            add(i, i);  
        }  
    }  
}
```

| Step | Component | What happens                 |
|------|-----------|------------------------------|
| 1    | Developer | Writes Calculator.java       |
| 2    | JDK       | Provides javac               |
| 3    | javac     | Compiles .java source        |
| 4    | Result    | Calculator.class is produced |
| 5    | JVM       | Loads the .class file        |
| 6    | JVM       | Verifies/links the class     |

|    |                           |                                                      |
|----|---------------------------|------------------------------------------------------|
| 7  | JVM                       | Creates/uses runtime data areas                      |
| 8  | Execution Engine          | Begins executing the bytecode                        |
| 9  | Interpreter/JVM execution | Code begins running                                  |
| 10 | JVM profiling             | Frequently executed code can be identified           |
| 11 | <b>JIT</b>                | Compiles suitable hot code                           |
| 12 | JIT                       | Applies runtime optimizations                        |
| 13 | CPU                       | Executes native machine instructions                 |
| 14 | JVM                       | Continues managing memory, threads, exceptions, etc. |

## 7. JDK vs JRE vs JVM vs JIT — Simple Analogy

| Java Component | Real-world analogy                                                                 |
|----------------|------------------------------------------------------------------------------------|
| JDK            | A complete <b>workshop</b> containing tools for building a machine                 |
| JRE            | The <b>environment</b> required to operate the finished machine                    |
| JVM            | The <b>engine</b> that actually performs the work                                  |
| JIT            | An <b>engine optimizer/tuner</b> that makes frequently performed operations faster |

## 8. The Key Difference in One Table

| Question                    | JDK                | JRE             | JVM         | JIT                     |
|-----------------------------|--------------------|-----------------|-------------|-------------------------|
| Develop Java?               | ✓ Yes              | ✗ No            | ✗ No        | ✗ No                    |
| Compile .java?              | ✓ Yes, using javac | ✗               | ✗           | ✗                       |
| Create .class?              | ✓                  | ✗               | ✗           | ✗                       |
| Run Java application?       | ✓                  | ✓               | ✓           | ✗ independently         |
| Execute bytecode?           | Through JVM        | Through JVM     | ✓           | As compiled native code |
| Compile hot bytecode?       | Through JVM/JIT    | Through JVM/JIT | Through JIT | ✓                       |
| Manage Java runtime memory? | Through JVM        | Through JVM     | ✓           | ✗                       |
| Garbage collection?         | Through JVM        | Through JVM     | ✓           | ✗                       |
| Optimize hot code?          | Through JVM/JIT    | Through JVM/JIT | Through JIT | ✓                       |
| Primary role                | Development        | Runtime         | Execution   | Optimization            |

### Final formula

JDK  
 =  
 Development Tools  
 +  
 Java Runtime

JRE  
 =

Runtime Environment  
+  
JVM  
+  
Runtime Libraries  
↓  
JVM  
=  
Bytecode Execution  
+  
Memory Management  
+  
Class Loading  
+  
Execution Engine  
↓  
JIT  
=  
Hot Bytecode  
→  
Optimized Native Machine Code

### **The easiest way to remember**

**JDK develops Java.**

**JRE provides the runtime environment.**

**JVM executes Java bytecode.**

**JIT makes frequently executed code faster by compiling it to native machine code.**

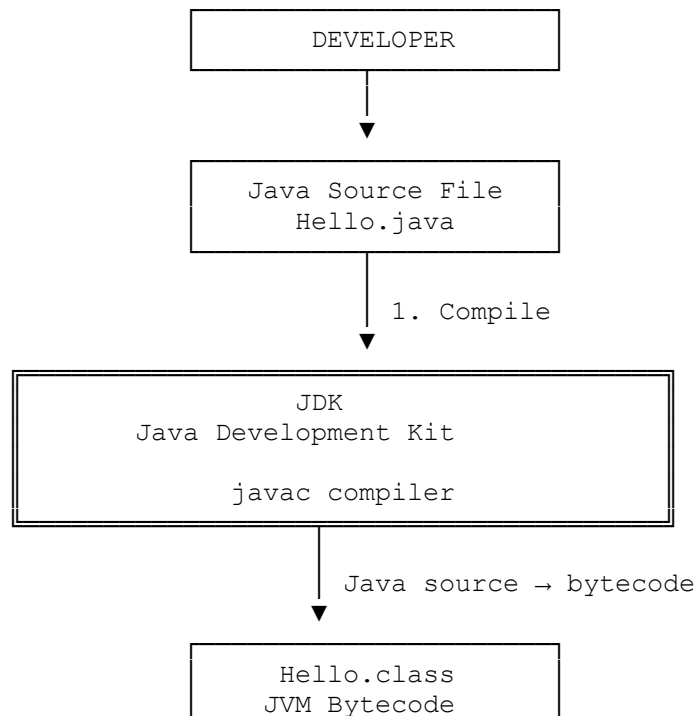
## Complete Java File Execution Flow — Start to End

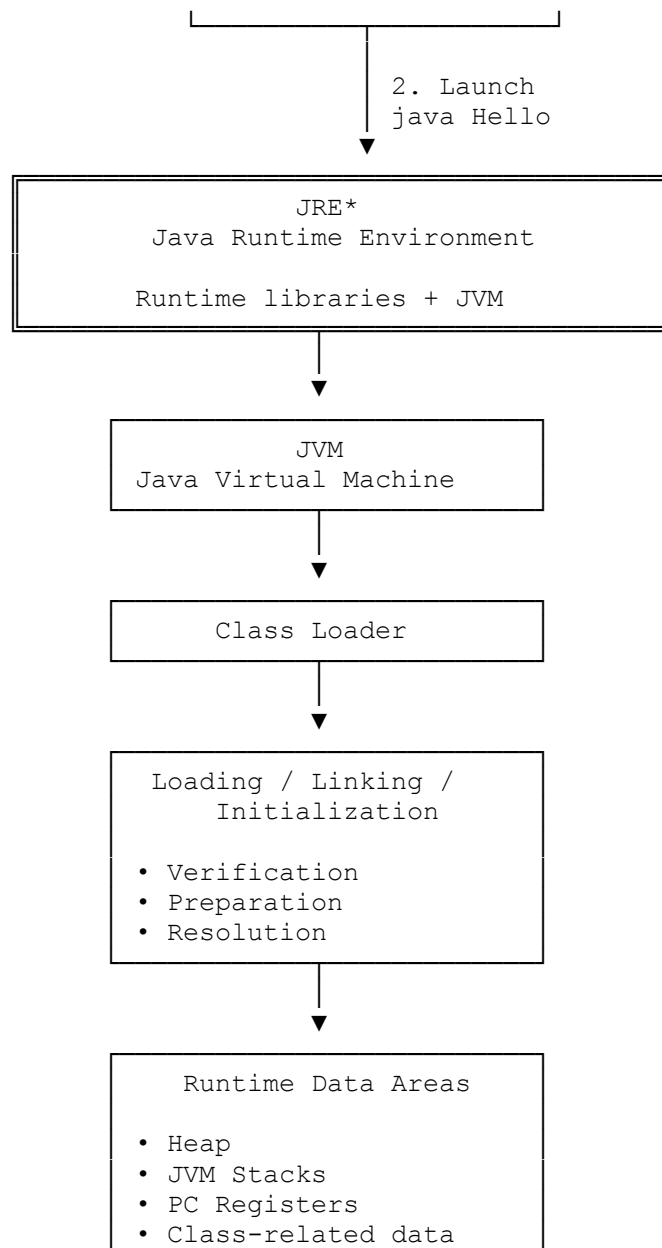
A Java program passes through **source-code compilation, class loading, verification/linking, runtime execution, JIT compilation/optimization, and CPU execution.**

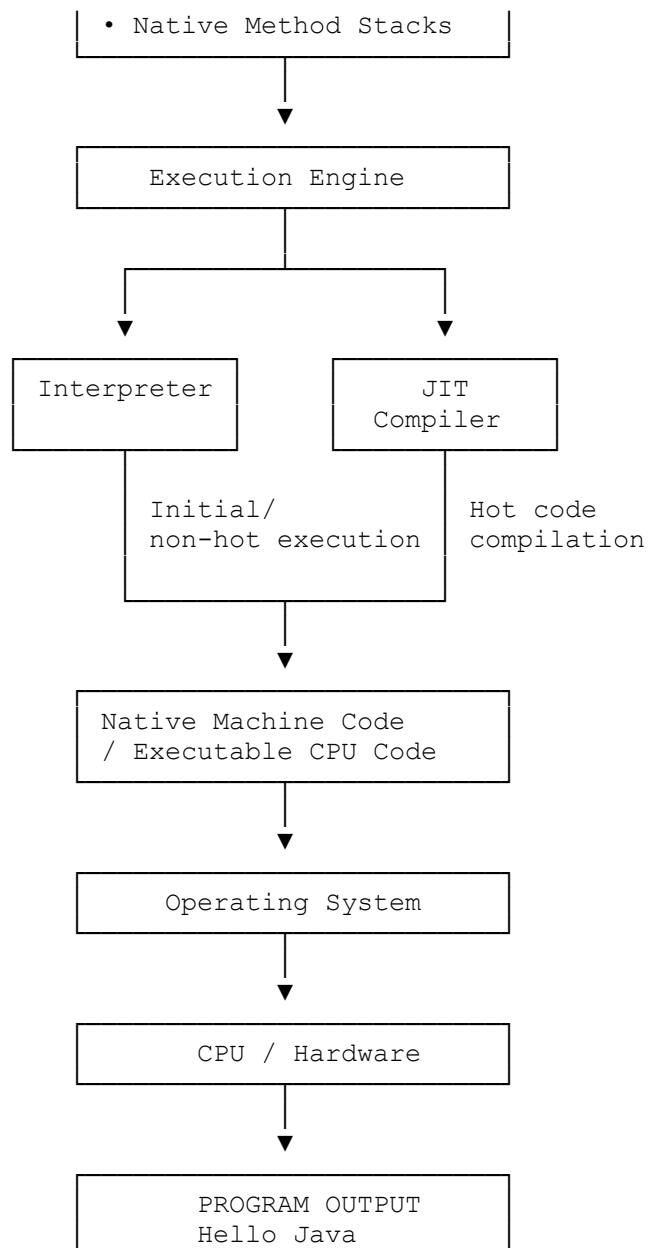
**Important:** The commonly taught sequence JDK → JRE → JVM → JIT is useful conceptually, but it is not literally a pipeline where a .java file travels through four separate programs. The **JDK provides development tools and a Java runtime**, the **JVM performs runtime execution**, and the **JIT is part of the JVM's execution machinery**. A separately installed JRE is mainly a Java 8-and-earlier packaging concept; modern Java uses modular runtime images.

---

### 1. Complete End-to-End Flowchart







**\*JRE is shown here as the traditional conceptual runtime layer. In modern Java, there is generally no separately installed JRE product in the old Java 8 sense.**

## 2. Complete Sequence in Table Form

| Step | Stage               | Component                          | Input                   | Output / Result                             |
|------|---------------------|------------------------------------|-------------------------|---------------------------------------------|
| 1    | Write source        | Developer                          | Java logic              | .java file                                  |
| 2    | Compile             | <b>JDK / javac</b>                 | .java                   | .class bytecode                             |
| 3    | Launch              | Java launcher                      | Class name/JAR          | JVM starts                                  |
| 4    | Runtime environment | <b>JRE concept</b>                 | Application             | Runtime libraries + JVM                     |
| 5    | Load                | <b>JVM Class Loader</b>            | .class                  | Class loaded into JVM                       |
| 6    | Link                | <b>JVM</b>                         | Loaded class            | Verified, prepared, resolved                |
| 7    | Initialize          | <b>JVM</b>                         | Class                   | Static initialization performed as required |
| 8    | Memory              | <b>JVM</b>                         | Classes/objects/threads | Runtime data areas used                     |
| 9    | Execute             | <b>Execution Engine</b>            | Bytecode                | Program starts executing                    |
| 10   | Initial execution   | <b>Interpreter / JVM execution</b> | Bytecode                | Instructions executed                       |

|           |                        |            |                     |                                         |
|-----------|------------------------|------------|---------------------|-----------------------------------------|
| <b>11</b> | Profile                | <b>JVM</b> | Running code        | Hot/frequently executed code identified |
| <b>12</b> | Compile hot code       | <b>JIT</b> | Hot bytecode        | Native machine code                     |
| <b>13</b> | Optimize               | <b>JIT</b> | Runtime information | Optimized machine code                  |
| <b>14</b> | Execute optimized code | <b>CPU</b> | Native instructions | Fast execution                          |
| <b>15</b> | Runtime management     | <b>JVM</b> | Objects/threads     | GC, exceptions, synchronization, etc.   |
| <b>16</b> | Finish                 | JVM/OS     | Program termination | Exit status/output                      |

---

### 3. Step 1 — Create the .java File

Suppose we create:

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello Java");
    }
}
```

Save it as:

HelloWorld.java

#### Definition

A **Java source file** is a human-readable text file containing Java source code.

HelloWorld.java



Java Source Code

At this stage:

- No JVM is executing the program.
- No JIT is compiling anything.
- The source is simply Java text.

---

#### 4. Step 2 — JDK Compiles the Source Code

Run:

```
javac HelloWorld.java
```

Here, javac is the Java compiler supplied by the **JDK**.

HelloWorld.java



HelloWorld.class

##### Definition of javac

javac converts valid Java source code into **JVM bytecode** stored in class files.

The result is:

HelloWorld.class

## Important distinction

The compiler does **not normally produce CPU-specific machine code**.

It produces:

Java Source  
↓  
JVM Bytecode

---

## 5. Step 3 — .class File Contains Bytecode

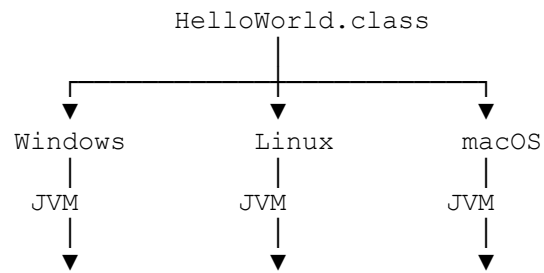
The .class file contains JVM class-file data, including bytecode instructions and metadata.

Conceptually:

```
HelloWorld.class
├── Class metadata
├── Constant-pool information
├── Method information
└── JVM bytecode
```

The bytecode is designed for a **JVM**, not directly for a particular CPU.

Therefore:



CPU

CPU

CPU

This is a major reason Java is portable.

---

## 6. Step 4 — Start the Application

Now execute:

```
java HelloWorld
```

The java launcher starts the JVM and asks it to launch the specified application.

Conceptually:

```
java HelloWorld
  |
  ▼
Start JVM
  |
  ▼
Load HelloWorld
```

Notice that you normally don't write:

```
java HelloWorld.class
```

Instead:

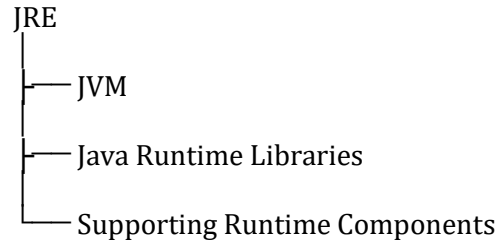
```
java HelloWorld
```

---

## 7. Step 5 — Runtime Environment

Traditionally, Java's runtime environment was described as the **JRE**.

Conceptually:



The runtime environment provides what the application needs to execute.

For example, your program uses:

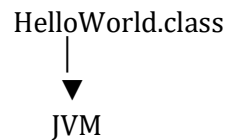
```
System.out.println(...)
```

The Java platform supplies the classes and runtime facilities needed by that operation.

---

## 8. Step 6 — JVM Starts

The **JVM (Java Virtual Machine)** now becomes the central execution component.



The JVM is responsible for:

- Loading classes
  - Linking classes
  - Initializing classes when required
  - Managing runtime memory
  - Managing threads
  - Executing bytecode
  - Handling exceptions
  - Supporting garbage collection
  - Interacting with native code
  - Using JIT compilation in JVM implementations that provide it
- 

## 9. Step 7 — Class Loading

The JVM's **Class Loader subsystem** loads the required classes.

HelloWorld.class



Class Loader



Loaded class

### Example

Your program has:

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello Java");  
    }  
}
```

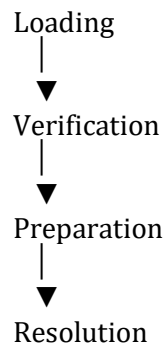
The JVM needs HelloWorld and other platform classes needed by the program.

The class-loading mechanism locates and loads class definitions as required.

---

## 10. Step 8 — Linking

After loading, the JVM performs the stages commonly described as **linking**:



### Verification

The JVM checks that the class-file structure and bytecode satisfy JVM constraints.

### Preparation

The JVM prepares memory associated with class/static fields and runtime structures.

### Resolution

Symbolic references can be resolved to appropriate runtime entities.

Resolution may occur lazily depending on the JVM implementation and circumstances.

---

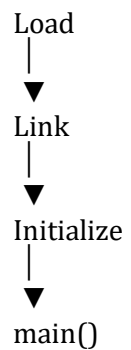
## 11. Step 9 — Class Initialization

When required, the JVM initializes a class.

For example:

```
public class Test {  
    static int number = 100;  
  
    static {  
        System.out.println("Class initialized");  
    }  
  
    public static void main(String[] args) {  
        System.out.println(number);  
    }  
}
```

Conceptually:



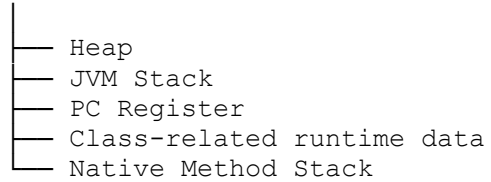
Static initialization occurs as part of the JVM's class-initialization rules.

---

## 12. Step 10 — JVM Runtime Memory

While the program runs, the JVM uses runtime data areas.

JVM Runtime Data Areas



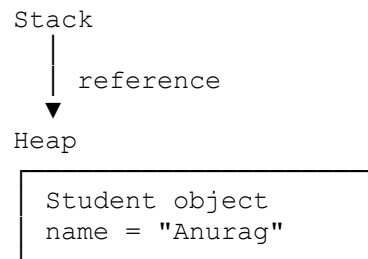
### Heap

Objects are generally allocated in the heap.

Example:

```
Student s = new Student();
```

Conceptually:



## JVM Stack

Method calls use stack frames.

```
main()  
  |  
  ▼  
calculate()  
  |  
  ▼  
printResult()
```

Conceptually:

JVM Stack

|               |
|---------------|
| printResult() |
| calculate()   |
| main()        |

Each Java thread has its own JVM stack.

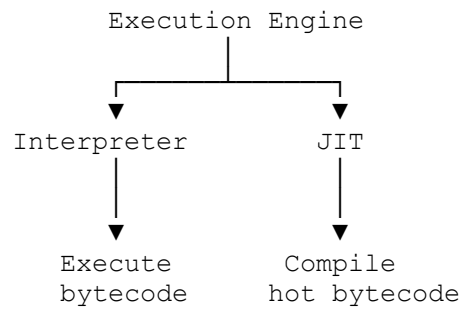
---

## 13. Step 11 — Execution Engine

The JVM's execution mechanisms execute the bytecode.

A simplified representation is:

```
JVM  
  |  
  ▼
```

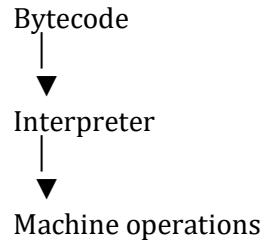


---

## 14. Step 12 — Interpreter

The JVM can initially execute bytecode through interpretation.

Conceptually:



For example, suppose bytecode represents an operation equivalent to:

```
int result = a + b;
```

The JVM's execution mechanisms process the bytecode and ultimately cause the corresponding computation to happen on the hardware.

---

## 15. Step 13 — JVM Identifies Hot Code

Consider:

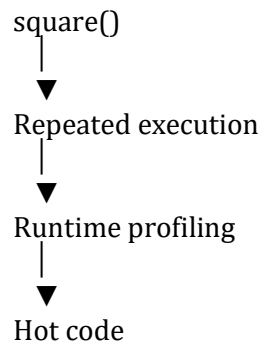
```
static int square(int x) {  
    return x * x;  
}  
  
public static void main(String[] args) {  
  
    long total = 0;  
  
    for (int i = 0; i < 100_000_000; i++) {  
        total += square(i);  
    }  
}
```

The method:

square()

may execute an enormous number of times.

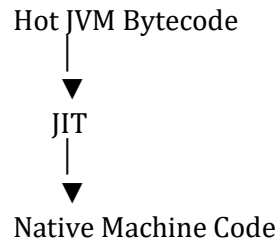
The JVM gathers runtime information and can identify frequently executed code paths as **hot code**.



---

## 16. Step 14 — JIT Compiles Hot Code

Now the **JIT (Just-In-Time Compiler)** can compile suitable hot bytecode into native machine code.



This is fundamentally different from javac.

| Compiler | Input        | Output              | When                         |
|----------|--------------|---------------------|------------------------------|
| jvac     | .java source | JVM bytecode        | Before application execution |
| JIT      | JVM bytecode | Native machine code | During application execution |

Therefore:

jvac:  
.java → .class

JIT:  
.class bytecode → native machine code

---

## 17. Step 15 — JIT Optimization

The JIT doesn't merely translate bytecode; it can use runtime information to optimize compiled code.

Depending on the JVM implementation and circumstances, optimizations can include:

| Optimization              | Basic idea                                                 |
|---------------------------|------------------------------------------------------------|
| Inlining                  | Replace suitable method calls with the method's operations |
| Dead-code elimination     | Remove code that cannot affect the result                  |
| Constant propagation      | Use known constant values to simplify calculations         |
| Escape analysis           | Determine whether objects escape a particular scope        |
| Devirtualization          | Optimize certain dynamic method dispatch situations        |
| Loop optimizations        | Improve frequently executed loops                          |
| CPU-specific optimization | Generate instructions suitable for the target processor    |

For example:

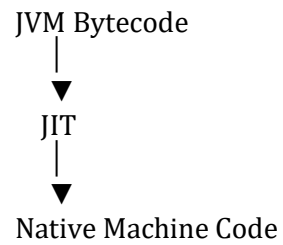
```
static int square(int x) {  
    return x * x;  
}
```

A JIT compiler may inline a suitable invocation into a hot caller, allowing further optimization.

---

## 18. Step 16 — Native Machine Code

After JIT compilation:



Native machine code is specific to the target architecture.

For example:

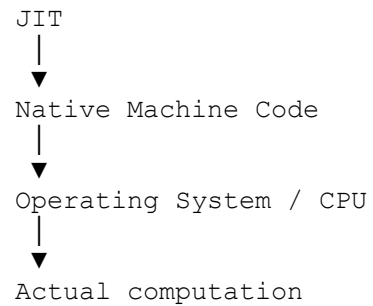
x86-64 machine code  
ARM64 machine code  
etc.

This is where the JVM bridges the gap between portable Java bytecode and the actual processor.

---

## 19. Step 17 — CPU Executes the Code

The operating system and hardware ultimately execute the native instructions.



For our example:

square(10)  
↓  
10 × 10  
↓  
100

---

## 20. Step 18 — Program Produces Output

For:

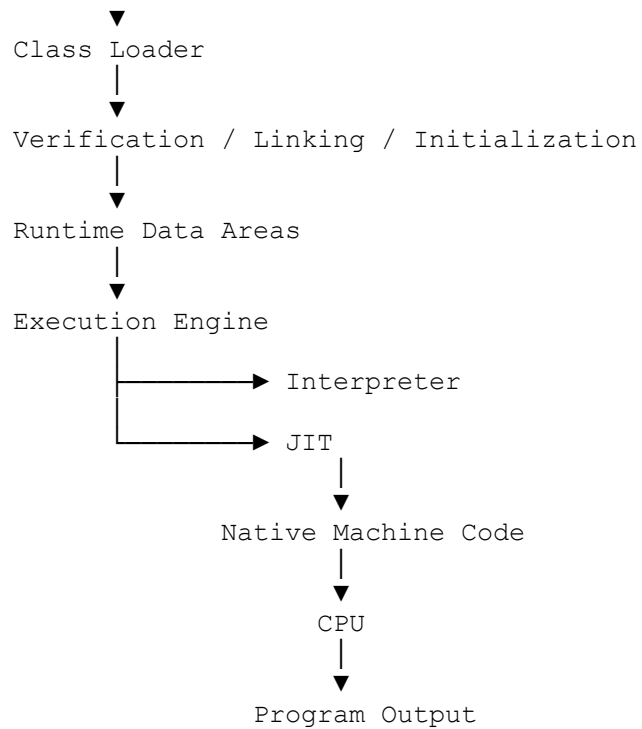
```
System.out.println("Hello Java");
```

the eventual observable result is:

Hello Java

The complete simplified path is:

HelloWorld.java  
↓  
JDK  
↓  
javac  
↓  
HelloWorld.class  
↓  
Runtime  
↓  
JVM  
↓



---

## 21. Where Each Component Fits

| Component | Position                             | What it does                                                         |
|-----------|--------------------------------------|----------------------------------------------------------------------|
| JDK       | Development stage + provides runtime | Supplies javac, java, jar, javadoc, debugging and other tools        |
| JRE       | Runtime concept                      | Provides the runtime environment needed to execute Java applications |
| JVM       | Main runtime execution component     | Loads and executes bytecode                                          |

|                    |                                   |                                                 |
|--------------------|-----------------------------------|-------------------------------------------------|
| Class Loader       | Inside JVM                        | Loads class definitions                         |
| Verifier/Linker    | JVM class-loading/linking process | Checks and prepares classes                     |
| Runtime Data Areas | Inside JVM                        | Provides memory/runtime structures              |
| Interpreter        | JVM execution machinery           | Executes bytecode                               |
| JIT                | JVM execution machinery           | Compiles hot bytecode into native code          |
| Garbage Collector  | JVM                               | Reclaims memory occupied by unreachable objects |
| OS                 | Below JVM                         | Provides system resources                       |
| CPU                | Hardware                          | Executes native machine instructions            |

## 22. Complete Example With javac and java

### Source

```
public class Calculator {

    static int add(int a, int b) {
        return a + b;
    }

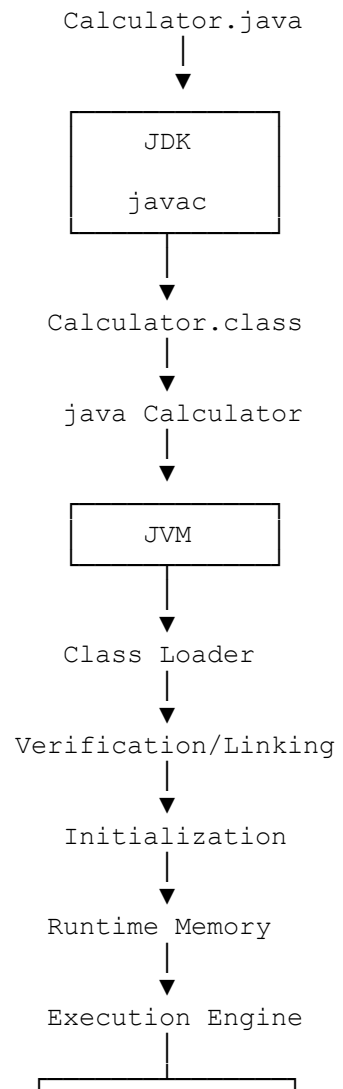
    public static void main(String[] args) {

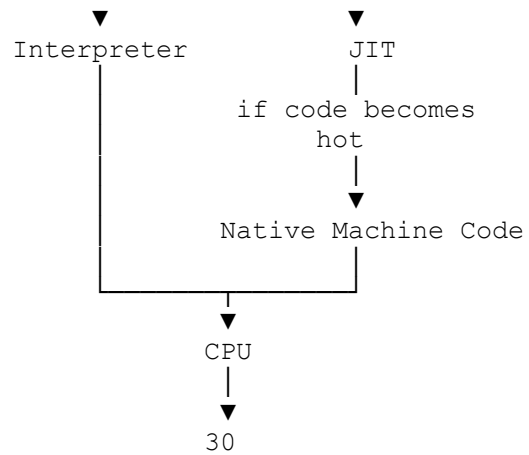
        int result = add(10, 20);

        System.out.println(result);
    }
}
```

}

## Execution





---

### 23. What Happens When the Program Is Very Short?

A common misunderstanding is:

"Every Java program must first be interpreted and then JIT compiled."

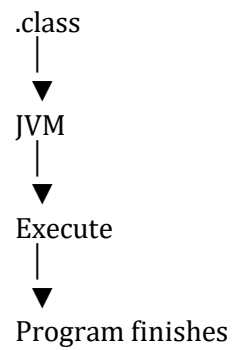
Not necessarily.

For a tiny program such as:

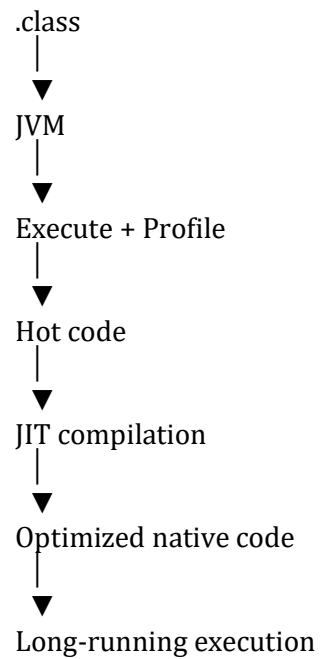
```
public class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello");  
    }  
}
```

there may be little opportunity for JIT compilation because the program can finish very quickly.

Conceptually:



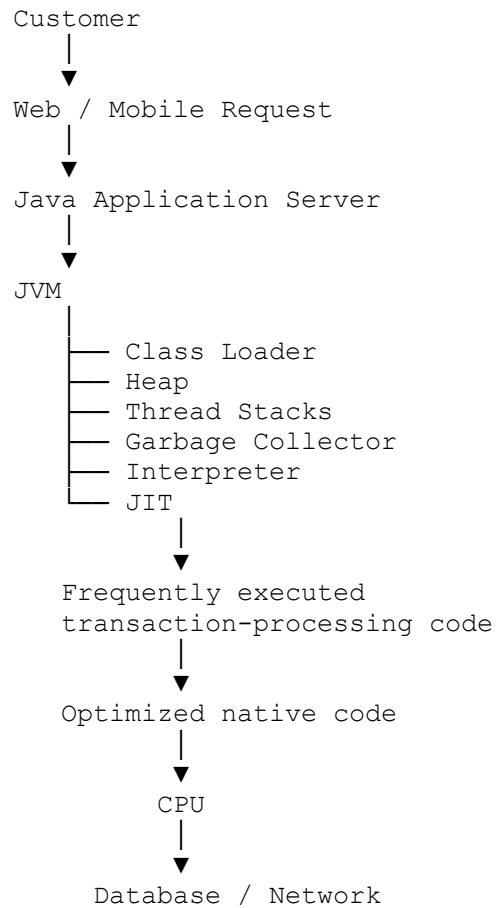
For a long-running application:



---

## 24. Real-Time Enterprise Example

Consider a Java banking application.



For a long-running banking service, the same request-processing methods may execute repeatedly. Runtime profiling and JIT compilation can therefore provide significant performance benefits.

---

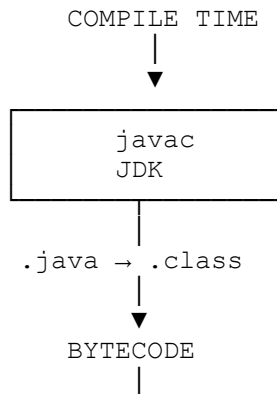
## 25. Complete Sequence — One Table

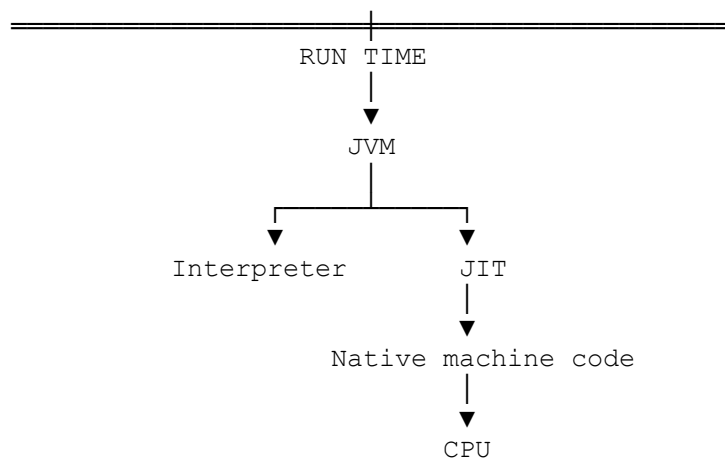
| #  | Event                       | Component        | Example                               |
|----|-----------------------------|------------------|---------------------------------------|
| 1  | Write code                  | Developer        | HelloWorld.java                       |
| 2  | Save source                 | File system      | .java                                 |
| 3  | Invoke compiler             | JDK              | javac HelloWorld.java                 |
| 4  | Parse source                | javac            | Syntax/semantic analysis              |
| 5  | Generate bytecode           | javac            | HelloWorld.class                      |
| 6  | Start application           | Java launcher    | java HelloWorld                       |
| 7  | Start runtime               | JVM              | JVM process starts                    |
| 8  | Find class                  | Class Loader     | Locate HelloWorld                     |
| 9  | Load class                  | JVM              | Class definition loaded               |
| 10 | Verify                      | JVM              | Check class-file/bytecode constraints |
| 11 | Prepare                     | JVM              | Prepare class-related runtime state   |
| 12 | Resolve                     | JVM              | Resolve symbolic references as needed |
| 13 | Initialize                  | JVM              | Run required class initialization     |
| 14 | Allocate runtime structures | JVM              | Heap, stack, etc.                     |
| 15 | Find main()                 | JVM              | public static void main(...)          |
| 16 | Begin execution             | Execution Engine | Execute bytecode                      |

|    |                           |                 |                             |
|----|---------------------------|-----------------|-----------------------------|
| 17 | Execute initial code      | Interpreter/JVM | Program starts              |
| 18 | Profile execution         | JVM             | Detect hot code             |
| 19 | Compile hot code          | JIT             | Bytecode → native code      |
| 20 | Optimize                  | JIT             | Runtime-based optimizations |
| 21 | Execute optimized code    | CPU             | Native instructions         |
| 22 | Manage memory             | JVM/GC          | Reclaim unreachable objects |
| 23 | Handle runtime events     | JVM/OS          | Exceptions, threads, I/O    |
| 24 | Finish application        | JVM/OS          | Exit                        |
| 25 | Release process resources | OS              | Process terminates          |

## 26. The Most Important Concept

There are **two different compilation stages** in the normal Java execution model:

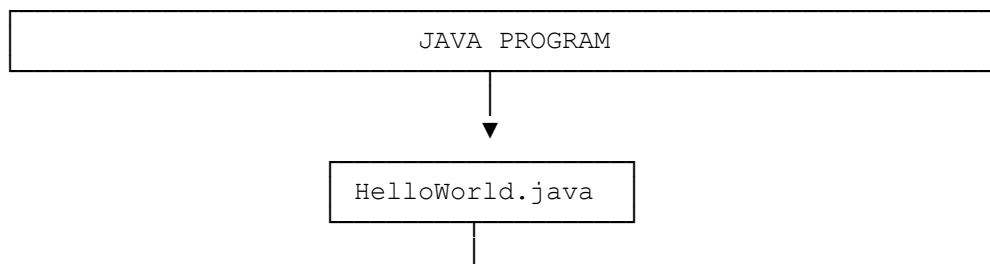


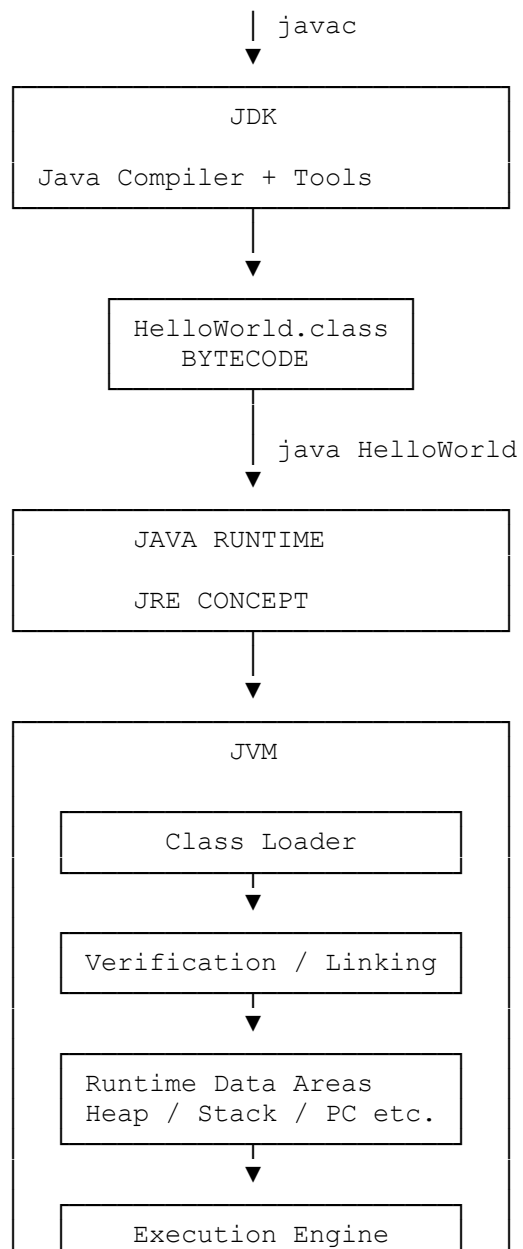


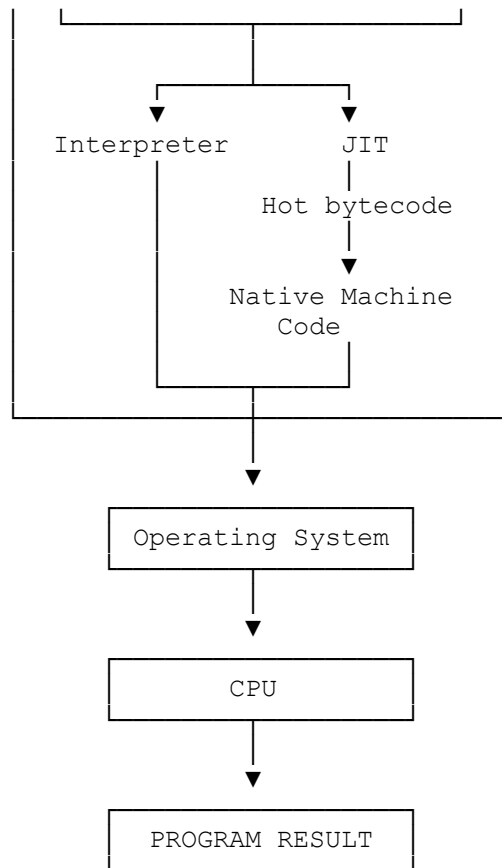
**Therefore:**

| Compilation      | Performed by   | Input        | Output              | Time             |
|------------------|----------------|--------------|---------------------|------------------|
| Java compilation | javac          | .java        | JVM bytecode .class | Before execution |
| JIT compilation  | JIT inside JVM | JVM bytecode | Native machine code | During execution |

## 27. Final Master Flowchart







**In one sentence**

**.java → JDK/javac → .class bytecode → JVM class loading/linking/initialization → runtime memory → execution engine → interpreter and/or JIT → native machine code → operating system → CPU → program output.**